5. RAG (Retrieval-Augmented Generation) и работа с документами

Фундаментальные принципы RAG

1.1. Концепция Retrieval-Augmented Generation

Определение 1.1. *RAG (Retrieval-Augmented Generation)* — это архитектура генеративных AI-систем, которая комбинирует поиск релевантной информации из внешней базы знаний с генерацией ответов на основе языковой модели, обеспечивая более точные, актуальные и обоснованные ответы

Мотивация создания RAG:

```
┌──────────────────────────────────────────────────────────────────────┐
│                      ПРОБЛЕМЫ ЧИСТЫХ LLM                             │
├──────────────────────────────────────────────────────────────────────┤
│                                                                      │
│  ❌ ГАЛЛЮЦИНАЦИИ                                                      │
│     LLM генерирует правдоподобную, но фактологически неверную информацию  │
│     Причина: Модель полагается только на знания из обучения          │
│                                                                      │
│  ❌ УСТАРЕВШИЕ ЗНАНИЯ                                                 │
│     Знания модели ограничены датой окончания обучения                │
│     Пример: GPT-4 обучена до сентября 2023                           │
│                                                                      │
│  ❌ ОТСУТСТВИЕ КОНТЕКСТА                                              │
│     Модель не знает о специфичных данных организации                 │
│     Нет доступа к внутренней документации                            │
│                                                                      │
│  ❌ НЕВОЗМОЖНОСТЬ ПРОВЕРКИ                                            │
│     Сложно отследить источник информации в ответе                    │
│     Нет цитирования источников                                       │
│                                                                      │
│  ✅ RAG РЕШАЕТ ЭТИ ПРОБЛЕМЫ:                                          │
│     • Доступ к актуальным данным                                     │
│     • Возможность цитирования источников                             │
│     • Снижение галлюцинаций                                          │
│     • Контекстуализация под домен                                    │
│                                                                      │
└──────────────────────────────────────────────────────────────────────┘
```

Математическая модель RAG:

```
P(answer | question) = Σ P(answer | question, documentᵢ) × P(documentᵢ | question)
                       i∈retrieved

где:
  • P(documentᵢ | question) — вероятность релевантности документа
  • P(answer | question, documentᵢ) — вероятность ответа при данном документе
  • retrieved — множество найденных релевантных документов (top-k)
```

Архитектурная схема RAG:

```
┌──────────────────────────────────────────────────────────────────────┐
│                        АРХИТЕКТУРА RAG СИСТЕМЫ                       │
├──────────────────────────────────────────────────────────────────────┤
│                                                                      │
│  ┌──────────────────────────────────────────────────────────────┐   │
│  │ ЭТАП 1: ПОДГОТОВКА ЗНАНИЙ (INDEXING PIPELINE)                │   │
│  │ ═══════════════════════════════════════════                  │   │
│  │                                                              │   │
│  │  ┌──────────┐    ┌──────────┐    ┌──────────┐    ┌──────────┐   │
│  │  │ Documents │──▶│ Chunking │──▶│ Embedding │──▶│ Vector   │   │
│  │  │  (PDF,   │    │ (Splitting)│   │ (Vector) │    │  Store   │   │
│  │  │   MD,    │    │          │    │          │    │ (Chroma) │   │
│  │  │   TXT)   │    │          │    │          │    │          │   │
│  │  └──────────┘    └──────────┘    └──────────┘    └──────────┘   │
│  │       │               │               │               │        │
│  │       ▼               ▼               ▼               ▼        │
│  │  Загрузка         Разбиение       Векторизация     Индексация  │
│  │  документов       на чанки        в embeddings     в векторной БД │
│  │                                                              │   │
│  └──────────────────────────────────────────────────────────────┘   │
│                                  │                                  │
│                                  ▼                                  │
│  ┌──────────────────────────────────────────────────────────────┐   │
```

```
│   │  ЭТАП 2: ВЫПОЛНЕНИЕ ЗАПРОСА (QUERY PIPELINE)                          │
│   │  ═══════════════════════════════════════════                           │
│   │                                                                        │
│   │  ┌──────────┐    ┌──────────┐    ┌──────────┐    ┌──────────┐        │
│   │  │   User   │──→│ Embedding │──→│  Similar  │──→│  Context  │        │
│   │  │  Query   │    │  (Query)  │    │  Search   │    │ Building  │        │
│   │  └──────────┘    └──────────┘    └──────────┘    └──────────┘        │
│   │       │                │                │                │             │
│   │       ▼                ▼                ▼                ▼             │
│   │  Вопрос用户       Векторизация      Поиск k          Формирование      │
│   │  (текст)           запроса          ближайших        контекста для LLM  │
│   │                                     документов                         │
│   │                                                                        │
│   └────────────────────────────────────────────────────────────────────────┘
│                                   │
│                                   ▼
│   ┌────────────────────────────────────────────────────────────────────────┐
│   │  ЭТАП 3: ГЕНЕРАЦИЯ ОТВЕТА (GENERATION PIPELINE)                        │
│   │  ═══════════════════════════════════════════                           │
│   │                                                                        │
│   │  ┌──────────────────────────────────────────────────────────────┐     │
│   │  │                    PROMPT CONSTRUCTION                       │     │
│   │  │  ┌──────────────────────────────────────────────────────────┐ │     │
│   │  │  │ System: Вы — полезный ассистент. Используйте контекст ниже. │ │     │
│   │  │  │ Context: [найденные документы]                           │ │     │
│   │  │  │ Question: [вопрос пользователя]                          │ │     │
│   │  │  │ Answer:                                                  │ │     │
│   │  │  └──────────────────────────────────────────────────────────┘ │     │
│   │  └──────────────────────────────────────────────────────────────┘     │
│   │                              │                                         │
│   │                              ▼                                         │
│   │  ┌──────────────────────────────────────────────────────────────┐     │
│   │  │                   LLM GENERATION (YandexGPT)                 │     │
│   │  └──────────────────────────────────────────────────────────────┘     │
│   │                              │                                         │
│   │                              ▼                                         │
│   │  ┌──────────────────────────────────────────────────────────────┐     │
│   │  │                  FINAL ANSWER WITH CITATIONS                 │     │
│   │  └──────────────────────────────────────────────────────────────┘     │
│   │                                                                        │
│   └────────────────────────────────────────────────────────────────────────┘
│                                                                            │
└────────────────────────────────────────────────────────────────────────────┘
```

1.1.1. Ключевые компоненты RAG системы

Таблица 1.1. Компоненты RAG и их назначение

Компонент	Назначение	Технологии	Параметры
Document Loader	Загрузка документов из различных источников	PyPDF2, Unstructured, LangChain	Форматы: PDF, MD, TXT, DOCX
Text Splitter	Разбиение текста на чанки оптимального размера	Recursive, Character, Token	Chunk size: 256-1024 токенов
Embedding Model	Преобразование текста в векторные представления	sentence-transformers, BGE	Размерность: 384-768
Vector Store	Хранение и поиск векторных embeddings	Chroma, Milvus, Qdrant	Метрика: cosine, L2, dot
Retriever	Поиск релевантных документов	Similarity search, MMR	Top-k: 3-10 документов
LLM	Генерация финального ответа	YandexGPT, GigaChat	Temperature: 0.3-0.7

1.1.2. Преимущества RAG перед fine-tuning

Таблица 1.2. Сравнение RAG и Fine-tuning

Критерий	RAG	Fine-tuning
Актуальность данных	✅ Мгновенное обновление	❌ Требуется переобучение
Стоимость	✅ Низкая (только API вызовы)	❌ Высокая (обучение модели)
Время внедрения	✅ Часы-дни	❌ Дни-недели
Точность фактов	✅ Высокая (источники)	⚠️ Зависит от данных обучения
Галлюцинации	✅ Снижены	⚠️ Могут сохраняться
Прозрачность	✅ Источники цитируются	❌ Чёрный ящик
Масштабируемость	✅ Легко добавить документы	❌ Требуется новое обучение
Рекомендация	Для работы с документами	Для изменения стиля/поведения

1.2. Векторные базы данных

1.2.1. Концепция векторного поиска

Векторный поиск* — это метод поиска наиболее похожих элементов в многомерном векторном пространстве, где сходство измеряется математическими метриками (косинусное сходство, евклидово расстояние)

Принцип работы:

```
┌──────────────────────────────────────────────────────────────────────┐
│                    ПРИНЦИП ВЕКТОРНОГО ПОИСКА                         │
├──────────────────────────────────────────────────────────────────────┤
│                                                                      │
│  ШАГ 1: ВЕКТОРИЗАЦИЯ ДОКУМЕНТОВ                                      │
│  ═══════════════════════════════════                                 │
│                                                                      │
│  Документ 1: "Искусственный интеллект — область компьютерных наук"   │
│              ↓ (Embedding Model)                                     │
│              [0.23, -0.45, 0.67, ..., 0.12]  (384 dimensions)        │
│                                                                      │
│  Документ 2: "Машинное обучение — подраздел ИИ"                      │
│              ↓ (Embedding Model)                                     │
│              [0.31, -0.38, 0.72, ..., 0.09]  (384 dimensions)        │
│                                                                      │
│  ШАГ 2: ВЕКТОРИЗАЦИЯ ЗАПРОСА                                         │
│  ═══════════════════════════════                                     │
│                                                                      │
│  Запрос: "Что такое искусственный интеллект?"                        │
│              ↓ (Embedding Model)                                     │
│              [0.25, -0.42, 0.69, ..., 0.11]  (384 dimensions)        │
│                                                                      │
│  ШАГ 3: ВЫЧИСЛЕНИЕ СХОДСТВА                                          │
│  ═══════════════════════════════                                     │
│                                                                      │
│  Cosine Similarity(query, doc1) = 0.95  ← Наиболее похож             │
│  Cosine Similarity(query, doc2) = 0.87                               │
│                                                                      │
│  ШАГ 4: ВОЗВРАТ РЕЗУЛЬТАТОВ (Top-k)                                  │
│  ═══════════════════════════════════                                 │
│                                                                      │
│  Результат: [Документ 1, Документ 2]  (отсортировано по сходству)    │
│                                                                      │
└──────────────────────────────────────────────────────────────────────┘
```

Формула косинусного сходства:

```
cosine_similarity(A, B) = (A · B) / (||A|| × ||B||)

где:
  • A · B — скалярное произведение векторов
  • ||A|| — евклидова норма вектора A
  • ||B|| — евклидова норма вектора B
  • Результат: от -1 (противоположные) до 1 (идентичные)
```

1.2.2. Обзор векторных баз данных (2026)

Таблица 1.3. Сравнение векторных БД

Векторная БД	Лицензия	Self-hosted	РФ поддержка	Размерность	Метрики	Рекомендация
Chroma	Apache 2.0	✅ Да	✅ Через Python	до 1536	cosine, L2	⭐⭐⭐⭐⭐
Milvus	Apache 2.0	✅ Да	✅ Есть документация	до 32768	все	⭐⭐⭐⭐
Qdrant	Apache 2.0	✅ Да	✅ Российская разработка	до 4096	все	⭐⭐⭐⭐⭐
Weaviate	BSD 3	✅ Да	⚠️ Ограничено	до 2048	все	⭐⭐⭐
FAISS	MIT	✅ Да	✅ Meta (доступен)	до 65536	все	⭐⭐⭐⭐
Pinecone	Проприетарная	❌ Нет	❌ Нет	до 2048	все	⭐⭐

Рекомендация для РФ: Chroma (простота) или Qdrant (российская разработка, производительность)

1.2.3. Архитектура ChromaDB

```
┌──────────────────────────────────────────────────────────────────────────┐
│                        АРХИТЕКТУРА CHROMADB                              │
├──────────────────────────────────────────────────────────────────────────┤
│                                                                          │
│  ┌──────────────────────────────────────────────────────────────────┐   │
│  │  CLIENT LAYER (Python/JS API)                                    │   │
│  │  • Collection management                                         │   │
│  │  • Add/Query/Delete operations                                   │   │
│  │  • Metadata filtering                                            │   │
│  └──────────────────────────────────────────────────────────────────┘   │
│                                  │                                       │
│                                  ▼                                       │
│  ┌──────────────────────────────────────────────────────────────────┐   │
│  │  CORE ENGINE                                                     │   │
│  │  ┌──────────────┐  ┌──────────────┐  ┌──────────────┐           │   │
│  │  │  Segment     │  │  Index       │  │  Query       │           │   │
│  │  │  Manager     │  │  Builder     │  │  Executor    │           │   │
│  │  └──────────────┘  └──────────────┘  └──────────────┘           │   │
│  └──────────────────────────────────────────────────────────────────┘   │
│                                  │                                       │
│                                  ▼                                       │
│  ┌──────────────────────────────────────────────────────────────────┐   │
│  │  STORAGE LAYER                                                   │   │
│  │  ┌──────────────┐  ┌──────────────┐  ┌──────────────┐           │   │
│  │  │  Embeddings  │  │  Metadata    │  │  Documents   │           │   │
│  │  │  (HNSW)      │  │  (SQLite)    │  │  (Binary)    │           │   │
│  │  └──────────────┘  └──────────────┘  └──────────────┘           │   │
│  └──────────────────────────────────────────────────────────────────┘   │
│                                                                          │
│  HNSW (Hierarchical Navigable Small World):                              │
│  • Алгоритм приближённого поиска ближайших соседей                       │
│  • Сложность: O(log N)                                                   │
│  • Точность: 95-99% от точного поиска                                    │
│                                                                          │
└──────────────────────────────────────────────────────────────────────────┘
```

1.3. Стратегии чанкинга (разбиения текста)

1.3.1. Методы разбиения документов

Таблица 1.4. Стратегии чанкинга

Метод	Описание	Размер чанка	Перекрытие	Когда использовать
Character	Разбиение по символам	Фиксированное	Опционально	Простые тексты
Recursive	Рекурсивное по разделителям	Гибкое	Да	Структурированные документы
Token	По токенам LLM	Точное	Да	Для согласования с LLM
Semantic	По смысловым границам	Вариативное	Да	Высокое качество поиска
Fixed-size	Фиксированный размер	Фиксированное	Нет	Быстрая обработка

Визуализация чанкинга с перекрытием:

```
┌──────────────────────────────────────────────────────────────────────────┐
│                    ЧАНКИНГ С ПЕРЕКРЫТИЕМ (OVERLAP)                       │
├──────────────────────────────────────────────────────────────────────────┤
│                                                                          │
│  Исходный документ (1000 токенов):                                       │
│  ═══════════════════════════════════                                     │
│  [Текст документа ..........................................................]  │
│                                                                          │
│  Разбиение на чанки (chunk_size=256, overlap=50):                        │
│  ═══════════════════════════════════════════════════                     │
│                                                                          │
│  Chunk 1: [0──────────────────────256]                                   │
│                    [206──────────────────────462]  Chunk 2                 │
│                              [412──────────────────────668]  Chunk 3        │
│                                        [618──────────────────────874]  Chunk 4 │
│                                                  [824──────────────────────1000] Chunk 5 │
│                                                                          │
│  Преимущества перекрытия:                                                │
│  • Сохранение контекста между чанками                                    │
```

```
│  • Улучшение качества поиска (ключевые слова не теряются на границах)      │
│  • Более точные embeddings                                                  │
│                                                                             │
│  Рекомендуемые параметры:                                                   │
│  • chunk_size: 256-512 токенов (для технических документов)                 │
│  • chunk_overlap: 50-100 токенов (20% от chunk_size)                        │
│                                                                             │
└─────────────────────────────────────────────────────────────────────────────┘
```

1.3.2. Выбор размера чанка

Эмпирические рекомендации:

```
┌─────────────────────────────────────────────────────────────────────────────┐
│                      ВЫБОР РАЗМЕРА ЧАНКА                                    │
├─────────────────────────────────────────────────────────────────────────────┤
│                                                                             │
│  ТИП ДОКУМЕНТА           │ РЕКОМЕНДУЕМЫЙ РАЗМЕР │ ПРИМЕЧАНИЯ              │
│  ═══════════════════════╪═══════════════════╪═══════════════════════       │
│  Технические мануалы     │ 512-1024 токена      │ Сохранять целостность   │
│  Юридические документы   │ 256-512 токена       │ Точные цитаты важны     │
│  Научные статьи          │ 512 токенов          │ По абзацам/разделам     │
│  FAQ / Вопросы-ответы    │ 128-256 токенов      │ Один вопрос = один чанк │
│  Код / Программы         │ 256-512 токенов      │ По функциям/классам     │
│  Диалоги / Чаты          │ 256 токенов          │ По репликам             │
│                                                                             │
│  ВЛИЯНИЕ НА КАЧЕСТВО:                                                       │
│  ═══════════════════                                                        │
│  • Слишком маленькие чанки (<128): теряется контекст                        │
│  • Слишком большие чанки (>1024): шум в embeddings, дороже API              │
│  • Оптимально: 256-512 токенов для большинства задач                        │
│                                                                             │
└─────────────────────────────────────────────────────────────────────────────┘
```

1.4. Embedding модели для русского языка

1.4.1. Рекомендуемые модели (2026)

Таблица 1.5. Embedding модели с поддержкой русского языка

Модель	Размерность	Языки	Лицензия	Качество	Скорость	Рекомендация
paraphrase-multilingual-MiniLM-L12-v2	384	50+	Apache 2.0	⭐⭐⭐⭐	⭐⭐⭐⭐⭐	⭐⭐⭐⭐⭐
rubert-tiny2	312	RU+EN	Apache 2.0	⭐⭐⭐⭐	⭐⭐⭐⭐⭐	⭐⭐⭐⭐⭐
bge-multilingual-gemma2	1024	100+	MIT	⭐⭐⭐⭐⭐	⭐⭐⭐	⭐⭐⭐⭐
intfloat/multilingual-e5-large	1024	100+	MIT	⭐⭐⭐⭐⭐	⭐⭐⭐	⭐⭐⭐⭐
DeepPavlov/rubert-base-cased	768	RU+EN	Apache 2.0	⭐⭐⭐⭐	⭐⭐⭐	⭐⭐⭐

Рекомендация для курса: `paraphrase-multilingual-MiniLM-L12-v2` (баланс качества и скорости)

1.4.2. Сравнение качества embeddings

```
┌─────────────────────────────────────────────────────────────────────────────┐
│                      СРАВНЕНИЕ КАЧЕСТВА EMBEDDINGS                          │
├─────────────────────────────────────────────────────────────────────────────┤
│                                                                             │
│  БЕНЧМАРК: MTEB (Massive Text Embedding Benchmark) — Russian subset         │
│  ═══════════════════════════════════════════════════════════════             │
│                                                                             │
│  Модель                              │ Retrieval │ STS │ Классификация     │
│  ═══════════════════════════════════╪═══════════╪═══╪═══════════════       │
│  bge-multilingual-gemma2             │    65.2   │ 82.1│      78.5         │
│  intfloat/multilingual-e5-large      │    64.8   │ 81.7│      77.9         │
│  paraphrase-multilingual-MiniLM-L12  │    61.5   │ 79.3│      75.2         │
│  rubert-tiny2                        │    58.7   │ 76.8│      72.1         │
│                                                                             │
│  ДЛЯ RAG ВАЖНЫ:                                                             │
│  ═══════════════                                                            │
│  1. Retrieval score — качество поиска релевантных документов                │
│  2. Скорость инференса — время векторизации                                 │
│  3. Размер модели — требования к памяти                                     │
```

```
│                                                                    │
│  ВЫБОР ДЛЯ КУРСА: paraphrase-multilingual-MiniLM-L12-v2            │
│  • Хорошее качество для русского языка                             │
│  • Быстрая векторизация                                            │
│  • Малый размер (420 MB)                                           │
│  • Стабильная работа в РФ                                          │
│                                                                    │
└────────────────────────────────────────────────────────────────────┘
```

1.5. Продвинутые техники RAG

1.5.1. Hybrid Search (гибридный поиск)

Определение 1.3. *Hybrid Search* — комбинация векторного поиска (semantic) и полнотекстового поиска (keyword) для улучшения качества результатов

```
┌────────────────────────────────────────────────────────────────────┐
│                    HYBRID SEARCH АРХИТЕКТУРА                       │
├────────────────────────────────────────────────────────────────────┤
│                                                                    │
│                          ┌──────────┐                              │
│                          │  Query   │                              │
│                          └──────────┘                              │
│                               │                                    │
│                ┌──────────────┼──────────────┐                     │
│                │              │              │                     │
│                ▼              ▼              ▼                     │
│      ┌──────────────┐ ┌──────────────┐ ┌──────────────┐            │
│      │ Vector Search │ │ Keyword Search│ │ Metadata Filter│           │
│      │ (Semantic)   │ │ (BM25)       │ │ (Date, Type) │           │
│      │ Weight: 0.6  │ │ Weight: 0.4  │ │ (Optional)   │           │
│      └──────┬───────┘ └──────┬───────┘ └──────┬───────┘            │
│             │                │                │                    │
│             └────────────────┼────────────────┘                    │
│                              │                                     │
│                              ▼                                     │
│                      ┌──────────────┐                              │
│                      │ Reciprocal Rank │                           │
│                      │ Fusion (RRF)  │                             │
│                      └──────┬───────┘                              │
│                             │                                      │
│                             ▼                                      │
│                      ┌──────────────┐                              │
│                      │ Final Results │                             │
│                      │ (Re-ranked)   │                             │
│                      └──────────────┘                              │
│                                                                    │
│  Преимущества:                                                     │
│  • Лучшее покрытие (semantic + keyword)                            │
│  • Точные совпадения терминов                                      │
│  • Устойчивость к разным формулировкам                             │
│                                                                    │
└────────────────────────────────────────────────────────────────────┘
```

1.5.2. RAG с переформулировкой запроса (Query Rewriting)

```
┌────────────────────────────────────────────────────────────────────┐
│                    QUERY REWRITING PIPELINE                        │
├────────────────────────────────────────────────────────────────────┤
│                                                                    │
│  Original Query: "Как настроить?"                                  │
│         │                                                          │
│         ▼                                                          │
│  ┌──────────────────────────────────────────────────────────────┐  │
│  │ LLM REWRITER (YandexGPT)                                     │  │
│  │ Prompt: "Переформулируй вопрос более конкретно для поиска..." │  │
│  └──────────────────────────────────────────────────────────────┘  │
│         │                                                          │
│         ▼                                                          │
│  Rewritten Queries:                                                │
│  • "Как настроить RAG систему на ChromaDB?"                        │
│  • "Инструкция по настройке векторной базы данных"                 │
│  • "RAG pipeline configuration steps"                              │
│         │                                                          │
```

```
│          ▼                                                          │
│   ┌──────────────────────────────────────────────────────────────┐  │
│   │  PARALLEL SEARCH (multiple queries)                          │  │
│   └──────────────────────────────────────────────────────────────┘  │
│          │                                                          │
│          ▼                                                          │
│   ┌──────────────────────────────────────────────────────────────┐  │
│   │  MERGE & RERANK RESULTS                                      │  │
│   └──────────────────────────────────────────────────────────────┘  │
│                                                                     │
│   Улучшение качества: +15–25% к точности поиска                     │
│                                                                     │
└─────────────────────────────────────────────────────────────────────┘
```

1.5.3. Contextual Compression

Определение 1.4. *Contextual Compression* — техника сжатия найденных документов перед передачей в LLM для оптимизации использования контекстного окна

```
┌─────────────────────────────────────────────────────────────────────┐
│                      CONTEXTUAL COMPRESSION                         │
├─────────────────────────────────────────────────────────────────────┤
│                                                                     │
│   Retrieved Documents (10 chunks × 512 tokens = 5120 tokens)        │
│          │                                                          │
│          ▼                                                          │
│   ┌──────────────────────────────────────────────────────────────┐  │
│   │  LLM COMPRESSOR                                              │  │
│   │  Prompt: "Извлеки только релевантную информацию для вопроса..."  │
│   └──────────────────────────────────────────────────────────────┘  │
│          │                                                          │
│          ▼                                                          │
│   Compressed Context (10 chunks × 128 tokens = 1280 tokens)         │
│   • Экономия токенов: 75%                                           │
│   • Снижение стоимости API: 75%                                     │
│   • Улучшение качества: меньше шума                                 │
│                                                                     │
└─────────────────────────────────────────────────────────────────────┘
```

1.6. Российская специфика RAG

1.6.1. Доступные компоненты для РФ

Таблица 1.6. Стек технологий для RAG в РФ

Компонент	Российское решение	Альтернатива	Статус
LLM API	YandexGPT, GigaChat	Ollama (локально)	✅ Доступно
Embeddings	sentence-transformers (локально)	Yandex Embeddings	✅ Доступно
Vector DB	Chroma, Qdrant (self-hosted)	Milvus	✅ Доступно
Framework	LangChain (open source)	LlamaIndex	✅ Доступно
Hosting	Yandex Cloud, VK Cloud	Selectel	✅ Доступно
Monitoring	Yandex Cloud Logging	Prometheus (self)	✅ Доступно

1.6.2. Требования 152-ФЗ для RAG систем

```
┌─────────────────────────────────────────────────────────────────────┐
│                      152-ФЗ ТРЕБОВАНИЯ ДЛЯ RAG                      │
├─────────────────────────────────────────────────────────────────────┤
│                                                                     │
│   ЕСЛИ ОБРАБАТЫВАЮТСЯ ПЕРСОНАЛЬНЫЕ ДАННЫЕ:                          │
│   ═══════════════════════════════════════                           │
│   • Хранение данных на территории РФ                                │
│   • Согласие субъекта на обработку                                  │
│   • Шифрование при передаче и хранении                              │
│   • Реестр обработок ПДн                                            │
│   • Уведомление Роскомнадзора                                       │
│                                                                     │
│   РЕКОМЕНДАЦИИ ДЛЯ RAG:                                             │
│   ═══════════════════════                                           │
```

```
│  • Self-hosted векторная БД (Chroma/Qdrant на российском сервере)          │
│  • Российский LLM API (YandexGPT/GigaChat)                                 │
│  • Локальная векторизация (sentence-transformers)                          │
│  • Audit logging всех запросов                                             │
│  • Анонимизация данных перед обработкой                                    │
│                                                                            │
└────────────────────────────────────────────────────────────────────────────┘
```

🔬 ЛАБОРАТОРНАЯ РАБОТА №5

Тема: Создание RAG-системы для работы с документами

📋 ПАСПОРТ ЛАБОРАТОРНОЙ РАБОТЫ

Параметр	Значение
Номер работы	5
Тема	Создание RAG-системы для работы с документами
Время выполнения	2 часа (аудиторно) + 6 часов (самостоятельно)
Формат сдачи	Pull Request в ветку `week5` основного репозитория
Пререквизиты	Успешное выполнение лабораторных работ №1-4
Ресурсы	Все сервисы доступны с территории РФ

🎯 ЦЕЛИ И ЗАДАЧИ

Цели работы

1. ✅ Понимать архитектуру RAG-систем и принципы работы
2. ✅ Реализовать пайплайн индексации документов
3. ✅ Настроить векторную базу данных ChromaDB
4. ✅ Интегрировать RAG с российским LLM API (YandexGPT/GigaChat)
5. ✅ Адаптировать RAG-систему под тематику дипломной работы

Задачи

№	Задача	Результат
1	Настроить зависимости LangChain + ChromaDB	requirements.txt
2	Реализовать загрузку и чанкинг документов	Загрузчик документов
3	Создать векторное хранилище	ChromaDB-хранилище
4	Реализовать поиск и генерацию ответов	RAG pipeline
5	Адаптировать под специальность	Тематико-специфичный RAG
6	Протестировать и задокументировать	Тесты + документация

📁 ТРЕБОВАНИЯ К СТРУКТУРЕ ПРОЕКТА

Обновление репозитория

Создать ветку `week5` в существующем репозитории:

```bash
# Переход в репозиторий
cd ~/ai-course-labs

# Создание ветки week5 от main
git checkout main
git pull origin main
git checkout -b week5

# Создание структуры для Week 5
mkdir -p week5/src/rag
```

```bash
mkdir -p week5/data/documents
mkdir -p week5/data/chroma_db
mkdir -p week5/notebooks
mkdir -p week5/tests
mkdir -p week5/docs
```

Итоговая структура week5/

```
week5/
├── .env.example                    # Шаблон переменных окружения
├── requirements.txt                # Зависимости Python
├── data/
│   ├── documents/                  # Исходные документы
│   │   ├── sample1.pdf
│   │   └── sample2.md
│   └── chroma_db/                  # Векторная БД (в .gitignore)
├── src/
│   ├── __init__.py
│   ├── rag/
│   │   ├── __init__.py
│   │   ├── document_loader.py      # Загрузка документов
│   │   ├── chunking.py             # Разбиение на чанки
│   │   ├── embeddings.py           # Векторизация
│   │   ├── vector_store.py         # ChromaDB интеграция
│   │   └── rag_pipeline.py         # Основной RAG пайплайн
│   └── utils/
│       └── helpers.py              # Вспомогательные функции
├── notebooks/
│   └── lab5_rag_demo.ipynb         # Демонстрация работы
├── tests/
│   └── test_rag.py                 # Тесты RAG-системы
└── docs/
    └── report.md                   # Отчёт по работе
```

🔧 ЧАСТЬ 1: НАСТРОЙКА ОКРУЖЕНИЯ

Шаг 1.1: Установка зависимостей

Создайте файл `week5/requirements.txt`:

```
# requirements.txt
# Зависимости для лабораторной работы №5
# Версии зафиксированы для воспроизводимости

# RAG Core
langchain>=0.1.0,<0.2.0
langchain-community>=0.0.10,<0.1.0
langchain-chroma>=0.1.0,<0.2.0

# Vector Database
chromadb>=0.4.0,<0.5.0

# Embeddings (мультиязычные)
sentence-transformers>=2.2.0,<3.0.0

# Document Processing
pypdf>=3.17.0,<4.0.0
python-docx>=1.1.0,<2.0.0
unstructured>=0.11.0,<0.12.0
markdown>=3.5.0,<4.0.0

# Utilities
python-dotenv>=1.0.0,<2.0.0
requests>=2.31.0,<3.0.0
pydantic>=2.0.0,<3.0.0

# Development
jupyterlab>=4.0.0,<5.0.0
pytest>=7.0.0,<8.0.0
black>=23.0.0
flake8>=6.0.0
```

```
# Yandex Cloud
yandexcloud>=0.240.0
```

Установка зависимостей:

```
cd week5
pip install -r requirements.txt
pip freeze > requirements.txt
```

Шаг 1.2: Настройка переменных окружения

Создайте файл `week5/.env.example`:

```
# .env.example
# Шаблон переменных окружения для RAG системы

# Yandex Cloud API
YANDEX_IAM_TOKEN=your_iam_token_here
YANDEX_FOLDER_ID=your_folder_id_here

# ChromaDB Settings
CHROMA_PERSIST_DIRECTORY=./data/chroma_db
CHROMA_COLLECTION_NAME=rag_documents

# Embedding Model
EMBEDDING_MODEL=paraphrase-multilingual-MiniLM-L12-v2

# RAG Settings
CHUNK_SIZE=512
CHUNK_OVERLAP=50
TOP_K_RESULTS=5

# GigaChat API (альтернатива)
GIGACHAT_CLIENT_ID=your_client_id_here
GIGACHAT_CLIENT_SECRET=your_client_secret_here
```

💻 ЧАСТЬ 2: РЕАЛИЗАЦИЯ RAG КОМПОНЕНТОВ

Шаг 2.1: Загрузка документов

Создайте файл `week5/src/rag/document_loader.py`:

```
# -*- coding: utf-8 -*-
"""
Модуль загрузки документов из различных источников
Лабораторная работа №5
Дисциплина: Искусственный интеллект

Автор: [ФИО]
Группа: [НОМЕР ГРУППЫ]
Дата: 2026
"""

import os
import logging
from typing import List, Optional, Dict, Any
from pathlib import Path
from langchain_community.document_loaders import (
    PyPDFLoader,
    TextLoader,
    UnstructuredMarkdownLoader,
    DirectoryLoader
)
from langchain_core.documents import Document

logger = logging.getLogger(__name__)

class DocumentLoader:
    """
    Загрузчик документов из различных форматов.

    Поддерживаемые форматы:
    • PDF — технические документы, мануалы
```

```
    • TXT — простые текстовые файлы
    • MD — Markdown документация
    • DOCX — документы Word

    Атрибуты:
        source_directory: Путь к директории с документами
        supported_extensions: Поддерживаемые расширения файлов
    """

    def __init__(self, source_directory: str):
        """
        Инициализация загрузчика документов.

        Args:
            source_directory: Путь к директории с документами
        """
        self.source_directory = Path(source_directory)
        self.supported_extensions = ['.pdf', '.txt', '.md', '.docx']

        if not self.source_directory.exists():
            logger.warning(f"Директория не существует: {source_directory}")
            self.source_directory.mkdir(parents=True, exist_ok=True)

        logger.info(f"DocumentLoader инициализирован: {source_directory}")

    def load_document(self, file_path: str) -> List[Document]:
        """
        Загрузка单个 документа.

        Args:
            file_path: Путь к файлу

        Returns:
            List[Document]: Список загруженных документов

        Raises:
            ValueError: Если формат файла не поддерживается
        """
        path = Path(file_path)

        if not path.exists():
            raise FileNotFoundError(f"Файл не найден: {file_path}")

        extension = path.suffix.lower()

        logger.info(f"Загрузка документа: {file_path}")

        try:
            if extension == '.pdf':
                loader = PyPDFLoader(str(path))
            elif extension == '.txt':
                loader = TextLoader(str(path), encoding='utf-8')
            elif extension == '.md':
                loader = UnstructuredMarkdownLoader(str(path))
            elif extension == '.docx':
                from langchain_community.document_loaders import Docx2txtLoader
                loader = Docx2txtLoader(str(path))
            else:
                raise ValueError(f"Неподдерживаемый формат: {extension}")

            documents = loader.load()

            # Добавление метаданных
            for doc in documents:
                doc.metadata['source'] = str(path)
                doc.metadata['file_name'] = path.name
                doc.metadata['file_type'] = extension

            logger.info(f"Загружено {len(documents)} чанков из {file_path}")

            return documents

        except Exception as e:
            logger.error(f"Ошибка загрузки {file_path}: {e}")
            raise

    def load_directory(self, pattern: str = "**/*") -> List[Document]:
```

```python
        """
        Загрузка всех документов из директории.

        Args:
            pattern: Шаблон поиска файлов

        Returns:
            List[Document]: Список всех загруженных документов
        """
        all_documents = []

        for ext in self.supported_extensions:
            file_pattern = f"{pattern}{ext}"
            files = list(self.source_directory.glob(file_pattern))

            logger.info(f"Найдено {len(files)} файлов с расширением {ext}")

            for file_path in files:
                try:
                    documents = self.load_document(str(file_path))
                    all_documents.extend(documents)
                except Exception as e:
                    logger.warning(f"Пропущен файл {file_path}: {e}")

        logger.info(f"Всего загружено {len(all_documents)} документов")

        return all_documents

    def get_statistics(self) -> Dict[str, Any]:
        """Получение статистики по документам."""
        stats = {
            "directory": str(self.source_directory),
            "files": {},
            "total_size_bytes": 0
        }

        for ext in self.supported_extensions:
            files = list(self.source_directory.glob(f"**/*{ext}"))
            stats["files"][ext] = len(files)
            stats["total_size_bytes"] += sum(f.stat().st_size for f in files)

        stats["total_size_mb"] = round(stats["total_size_bytes"] / (1024 * 1024), 2)

        return stats


# Точка входа для тестирования
if __name__ == "__main__":
    from dotenv import load_dotenv
    load_dotenv()

    print("=" * 80)
    print("ТЕСТИРОВАНИЕ DOCUMENT LOADER")
    print("=" * 80)

    loader = DocumentLoader(source_directory="./data/documents")

    stats = loader.get_statistics()
    print(f"\nСтатистика директории:")
    print(f"  Директория: {stats['directory']}")
    print(f"  Общий размер: {stats['total_size_mb']} MB")
    print(f"  Файлы по типам: {stats['files']}")

    documents = loader.load_directory()
    print(f"\nЗагружено документов: {len(documents)}")

    if documents:
        print(f"\nПример первого документа:")
        print(f"  Источник: {documents[0].metadata.get('source')}")
        print(f"  Размер: {len(documents[0].page_content)} символов")
        print(f"  Предпросмотр: {documents[0].page_content[:200]}...")
```

Шаг 2.2: Разбиение на чанки

Создайте файл `week5/src/rag/chunking.py`:

```python
# -*- coding: utf-8 -*-
"""
Модуль разбиения документов на чанки
Лабораторная работа №5
"""

import logging
from typing import List, Optional
from langchain_text_splitters import (
    RecursiveCharacterTextSplitter,
    CharacterTextSplitter,
    TokenTextSplitter
)
from langchain_core.documents import Document

logger = logging.getLogger(__name__)

class ChunkingStrategy:
    """
    Стратегии разбиения текста на чанки.

    Поддерживаемые методы:
    • Recursive — рекурсивное по разделителям (рекомендуется)
    • Character — по символам фиксированного размера
    • Token — по токенам LLM
    """

    @staticmethod
    def get_splitter(
        strategy: str = "recursive",
        chunk_size: int = 512,
        chunk_overlap: int = 50
    ):
        """
        Получение сплиттера текста.

        Args:
            strategy: Стратегия разбиения (recursive/character/token)
            chunk_size: Размер чанка в символах/токенах
            chunk_overlap: Перекрытие между чанками

        Returns:
            TextSplitter: Настроенный сплиттер
        """
        logger.info(f"Создание сплиттера: {strategy}, chunk_size={chunk_size}")

        if strategy == "recursive":
            return RecursiveCharacterTextSplitter(
                separators=["\n\n", "\n", ". ", " ", ""],
                chunk_size=chunk_size,
                chunk_overlap=chunk_overlap,
                length_function=len,
                is_separator_regex=False
            )

        elif strategy == "character":
            return CharacterTextSplitter(
                separator="\n",
                chunk_size=chunk_size,
                chunk_overlap=chunk_overlap,
                length_function=len
            )

        elif strategy == "token":
            return TokenTextSplitter(
                chunk_size=chunk_size,
                chunk_overlap=chunk_overlap
            )

        else:
            raise ValueError(f"Неизвестная стратегия: {strategy}")

    @staticmethod
    def split_documents(
        documents: List[Document],
        strategy: str = "recursive",
        chunk_size: int = 512,
```

```python
        chunk_overlap: int = 50
    ) -> List[Document]:
        """
        Разбиение документов на чанки.

        Args:
            documents: Список документов для разбиения
            strategy: Стратегия разбиения
            chunk_size: Размер чанка
            chunk_overlap: Перекрытие

        Returns:
            List[Document]: Список чанков
        """
        splitter = ChunkingStrategy.get_splitter(
            strategy=strategy,
            chunk_size=chunk_size,
            chunk_overlap=chunk_overlap
        )

        logger.info(f"Разбиение {len(documents)} документов на чанки")

        chunks = splitter.split_documents(documents)

        logger.info(f"Получено {len(chunks)} чанков")

        # Добавление метаданных о чанках
        for i, chunk in enumerate(chunks):
            chunk.metadata['chunk_id'] = i
            chunk.metadata['total_chunks'] = len(chunks)

        return chunks

    @staticmethod
    def get_statistics(chunks: List[Document]) -> dict:
        """Статистика чанков."""
        if not chunks:
            return {"count": 0}

        sizes = [len(chunk.page_content) for chunk in chunks]

        return {
            "count": len(chunks),
            "min_size": min(sizes),
            "max_size": max(sizes),
            "avg_size": round(sum(sizes) / len(sizes), 2),
            "total_characters": sum(sizes)
        }


# Тестирование
if __name__ == "__main__":
    from rag.document_loader import DocumentLoader

    print("=" * 80)
    print("ТЕСТИРОВАНИЕ CHUNKING")
    print("=" * 80)

    loader = DocumentLoader(source_directory="./data/documents")
    documents = loader.load_directory()

    if documents:
        chunks = ChunkingStrategy.split_documents(
            documents,
            strategy="recursive",
            chunk_size=512,
            chunk_overlap=50
        )

        stats = ChunkingStrategy.get_statistics(chunks)
        print(f"\nСтатистика чанков:")
        print(f"  Количество: {stats['count']}")
        print(f"  Мин. размер: {stats['min_size']} символов")
        print(f"  Макс. размер: {stats['max_size']} символов")
        print(f"  Сред. размер: {stats['avg_size']} символов")
```

Шаг 2.3: Векторизация и хранение

Создайте файл `week5/src/rag/vector_store.py`:

```python
# -*- coding: utf-8 -*-
"""
Модуль работы с векторной базой данных ChromaDB
Лабораторная работа №5
"""

import os
import logging
from typing import List, Optional, Dict, Any
from langchain_chroma import Chroma
from langchain_community.embeddings import HuggingFaceEmbeddings
from langchain_core.documents import Document
import chromadb

logger = logging.getLogger(__name__)

class VectorStoreManager:
    """
    Менеджер векторной базы данных ChromaDB.

    Атрибуты:
        persist_directory: Путь для сохранения данных
        collection_name: Имя коллекции
        embedding_model: Модель для embeddings
    """

    def __init__(
        self,
        persist_directory: str = "./data/chroma_db",
        collection_name: str = "rag_documents",
        embedding_model: str = "paraphrase-multilingual-MiniLM-L12-v2"
    ):
        """
        Инициализация векторного хранилища.

        Args:
            persist_directory: Путь для сохранения данных
            collection_name: Имя коллекции
            embedding_model: Модель embeddings
        """
        self.persist_directory = persist_directory
        self.collection_name = collection_name
        self.embedding_model_name = embedding_model

        # Создание директории
        os.makedirs(persist_directory, exist_ok=True)

        # Инициализация embedding модели
        logger.info(f"Загрузка embedding модели: {embedding_model}")
        self.embeddings = HuggingFaceEmbeddings(
            model_name=f"sentence-transformers/{embedding_model}",
            model_kwargs={'device': 'cpu'},
            encode_kwargs={'normalize_embeddings': True}
        )

        # Инициализация ChromaDB
        logger.info(f"Инициализация ChromaDB: {collection_name}")
        self.vectorstore = Chroma(
            collection_name=collection_name,
            embedding_function=self.embeddings,
            persist_directory=persist_directory
        )

        logger.info("VectorStoreManager инициализирован")

    def add_documents(
        self,
        documents: List[Document],
        batch_size: int = 100
    ) -> Dict[str, Any]:
        """
        Добавление документов в векторную базу.

        Args:
            documents: Список документов для добавления
```

```
        batch_size: Размер пакета для добавления

    Returns:
        Dict: Статистика добавления
    """
    logger.info(f"Добавление {len(documents)} документов в векторную базу")

    # Добавление по пакетам для стабильности
    for i in range(0, len(documents), batch_size):
        batch = documents[i:i + batch_size]
        self.vectorstore.add_documents(batch)
        logger.info(f"Добавлен пакет {i//batch_size + 1}")

    stats = {
        "documents_added": len(documents),
        "collection_name": self.collection_name,
        "total_documents": self.vectorstore._collection.count()
    }

    logger.info(f"Всего документов в базе: {stats['total_documents']}")

    return stats

def search(
    self,
    query: str,
    k: int = 5,
    filter_metadata: Optional[Dict] = None
) -> List[Dict[str, Any]]:
    """
    Поиск релевантных документов.

    Args:
        query: Поисковый запрос
        k: Количество результатов
        filter_meta Фильтр по метаданным

    Returns:
        List[Dict]: Найденные документы с метаданными
    """
    logger.info(f"Поиск по запросу: {query[:100]}... (k={k})")

    docs = self.vectorstore.similarity_search(
        query=query,
        k=k,
        filter=filter_metadata
    )

    results = []
    for i, doc in enumerate(docs):
        results.append({
            "rank": i + 1,
            "content": doc.page_content,
            "metadata": doc.metadata,
            "similarity_score": doc.metadata.get('score', None)
        })

    logger.info(f"Найдено {len(results)} документов")

    return results

def search_with_scores(
    self,
    query: str,
    k: int = 5
) -> List[Dict[str, Any]]:
    """
    Поиск с оценками сходства.

    Args:
        query: Поисковый запрос
        k: Количество результатов

    Returns:
        List[Dict]: Документы с оценками
    """
    docs_and_scores = self.vectorstore.similarity_search_with_score(
```

```python
            query=query,
            k=k
        )

        results = []
        for i, (doc, score) in enumerate(docs_and_scores):
            results.append({
                "rank": i + 1,
                "content": doc.page_content,
                "metadata": doc.metadata,
                "similarity_score": 1 - score  # Преобразование в сходство
            })

        return results

    def delete_collection(self) -> bool:
        """Удаление коллекции."""
        try:
            client = chromadb.Client(
                chromadb.config.Settings(
                    persist_directory=self.persist_directory
                )
            )
            client.delete_collection(self.collection_name)
            logger.info(f"Коллекция {self.collection_name} удалена")
            return True
        except Exception as e:
            logger.error(f"Ошибка удаления: {e}")
            return False

    def get_statistics(self) -> Dict[str, Any]:
        """Статистика векторного хранилища."""
        return {
            "collection_name": self.collection_name,
            "persist_directory": self.persist_directory,
            "embedding_model": self.embedding_model_name,
            "total_documents": self.vectorstore._collection.count(),
            "embedding_dimension": 384
        }

    def clear(self) -> bool:
        """Очистка хранилища."""
        try:
            self.vectorstore._collection.delete(where={})
            logger.info("Хранилище очищено")
            return True
        except Exception as e:
            logger.error(f"Ошибка очистки: {e}")
            return False


# Тестирование
if __name__ == "__main__":
    from dotenv import load_dotenv
    from rag.document_loader import DocumentLoader
    from rag.chunking import ChunkingStrategy

    load_dotenv()

    print("=" * 80)
    print("ТЕСТИРОВАНИЕ VECTOR STORE")
    print("=" * 80)

    # Загрузка и чанкинг
    loader = DocumentLoader(source_directory="./data/documents")
    documents = loader.load_directory()

    if documents:
        chunks = ChunkingStrategy.split_documents(documents)

        # Векторное хранилище
        store = VectorStoreManager(
            persist_directory="./data/chroma_db",
            collection_name="test_collection"
        )

        # Добавление
```

```
        stats = store.add_documents(chunks)
        print(f"\nДобавлено документов: {stats['documents_added']}")

        # Поиск
        query = "искусственный интеллект"
        results = store.search(query, k=3)

        print(f"\nРезультаты поиска по запросу: {query}")
        for r in results:
            print(f"\n[{r['rank']}] Score: {r['similarity_score']:.3f}")
            print(f"Источник: {r['metadata'].get('source', 'N/A')}")
            print(f"Текст: {r['content'][:200]}...")

        # Статистика
        store_stats = store.get_statistics()
        print(f"\nСтатистика хранилища:")
        print(f"  Всего документов: {store_stats['total_documents']}")
        print(f"  Размерность embeddings: {store_stats['embedding_dimension']}")
```

Шаг 2.4: RAG пайплайн

Создайте файл `week5/src/rag/rag_pipeline.py`:

```
# -*- coding: utf-8 -*-
"""
Основный RAG пайплайн для генерации ответов
Лабораторная работа №5
Дисциплина: Искусственный интеллект

Автор: [ФИО]
Группа: [НОМЕР ГРУППЫ]
Дата: 2026
"""

import os
import logging
from typing import List, Dict, Any, Optional
from datetime import datetime
import time

from langchain.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser
from langchain_core.runnables import RunnablePassthrough

from rag.vector_store import VectorStoreManager

logger = logging.getLogger(__name__)

class RAGPipeline:
    """
    RAG пайплайн для генерации ответов на вопросы.

    Архитектура:
        1. Получение вопроса от пользователя
        2. Поиск релевантных документов в векторной базе
        3. Формирование промпта с контекстом
        4. Генерация ответа через LLM (YandexGPT)
        5. Возврат ответа с цитированием источников

    Атрибуты:
        vectorstore: Векторное хранилище
        llm: LLM клиент
        top_k: Количество документов для поиска
    """

    def __init__(
        self,
        vectorstore: VectorStoreManager,
        llm=None,
        top_k: int = 5
    ):
        """
        Инициализация RAG пайплайна.

        Args:
            vectorstore: Векторное хранилище
            llm: LLM клиент (YandexGPT/GigaChat)
```

```
            top_k: Количество документов для поиска
        """
        self.vectorstore = vectorstore
        self.llm = llm
        self.top_k = top_k

        # RAG промпт
        self.rag_prompt = ChatPromptTemplate.from_template("""
Ты — полезный ассистент для ответов на вопросы по документации.
Используй ТОЛЬКО предоставленный контекст для ответа.
Если ответа нет в контексте, скажи "В предоставленных документах нет информации по этому вопросу".

Контекст из документов:
{context}

Вопрос: {question}

Ответ:
""")

        logger.info("RAGPipeline инициализирован")

    def _format_context(self, documents: List[Dict]) -> str:
        """
        Форматирование контекста из найденных документов.

        Args:
            documents: Список найденных документов

        Returns:
            str: Форматированный контекст
        """
        formatted = []
        for i, doc in enumerate(documents, 1):
            source = doc.get('metadata', {}).get('source', 'Unknown')
            content = doc.get('content', '')
            formatted.append(f"[Источник {i}: {source}]\n{content}")

        return "\n\n".join(formatted)

    def query(
        self,
        question: str,
        include_sources: bool = True
    ) -> Dict[str, Any]:
        """
        Выполнение запроса к RAG системе.

        Args:
            question: Вопрос пользователя
            include_sources: Включать ли источники в ответ

        Returns:
            Dict: Ответ с метаданными
        """
        start_time = time.time()

        logger.info(f"RAG запрос: {question[:100]}...")

        # Шаг 1: Поиск релевантных документов
        search_results = self.vectorstore.search_with_scores(
            query=question,
            k=self.top_k
        )

        if not search_results:
            return {
                "success": False,
                "answer": "Не найдено релевантных документов в базе знаний.",
                "sources": [],
                "execution_time": 0
            }

        # Шаг 2: Формирование контекста
        context = self._format_context(search_results)

        # Шаг 3: Генерация ответа через LLM
```

```python
        if self.llm:
            prompt = self.rag_prompt.format(
                context=context,
                question=question
            )

            try:
                response = self.llm.generate(prompt, temperature=0.3)
                answer = response.get("text", "")
            except Exception as e:
                logger.error(f"Ошибка генерации: {e}")
                answer = f"Ошибка генерации ответа: {e}"
        else:
            # Fallback без LLM
            answer = f"Найдено {len(search_results)} релевантных документов.\n\n"
            for i, doc in enumerate(search_results, 1):
                answer += f"{i}. {doc['content'][:300]}...\n\n"

        # Шаг 4: Формирование результата
        execution_time = time.time() - start_time

        result = {
            "success": True,
            "question": question,
            "answer": answer,
            "sources": search_results if include_sources else [],
            "sources_count": len(search_results),
            "execution_time": round(execution_time, 3),
            "timestamp": datetime.now().isoformat()
        }

        logger.info(f"RAG ответ сгенерирован за {execution_time:.3f}с")

        return result

    def query_batch(
        self,
        questions: List[str]
    ) -> List[Dict[str, Any]]:
        """
        Пакетная обработка запросов.

        Args:
            questions: Список вопросов

        Returns:
            List[Dict]: Список ответов
        """
        results = []

        for i, question in enumerate(questions, 1):
            logger.info(f"Обработка запроса {i}/{len(questions)}")
            result = self.query(question)
            results.append(result)

        return results

    def get_statistics(self) -> Dict[str, Any]:
        """Статистика RAG пайплайна."""
        return {
            "top_k": self.top_k,
            "llm_enabled": self.llm is not None,
            "vectorstore_stats": self.vectorstore.get_statistics()
        }


# Точка входа для тестирования
if __name__ == "__main__":
    from dotenv import load_dotenv
    from langchain_community.llms import YandexGPT

    load_dotenv()

    print("=" * 80)
    print("ЛАБОРАТОРНАЯ РАБОТА №5")
    print("ТЕСТИРОВАНИЕ RAG PIPELINE")
    print("=" * 80)
```

```python
# Инициализация компонентов
from rag.vector_store import VectorStoreManager

vectorstore = VectorStoreManager(
    persist_directory="./data/chroma_db",
    collection_name="rag_documents"
)

# YandexGPT
iam_token = os.getenv("YANDEX_IAM_TOKEN")
folder_id = os.getenv("YANDEX_FOLDER_ID")

llm = None
if iam_token and folder_id:
    llm = YandexGPT(
        iam_token=iam_token,
        folder_id=folder_id,
        temperature=0.3,
        max_tokens=500
    )
    print("✅ YandexGPT подключён")
else:
    print("⚠️ YandexGPT не подключён (работаем без LLM)")

# RAG пайплайн
rag = RAGPipeline(
    vectorstore=vectorstore,
    llm=llm,
    top_k=5
)

# Тестовые запросы
test_questions = [
    "Что такое искусственный интеллект?",
    "Как работает RAG система?",
    "Какие есть векторные базы данных?"
]

print("\n" + "=" * 80)
print("ТЕСТОВЫЕ ЗАПРОСЫ")
print("=" * 80)

for i, question in enumerate(test_questions, 1):
    print(f"\n{'='*60}")
    print(f"ВОПРОС {i}: {question}")
    print(f"{'='*60}")

    result = rag.query(question)

    print(f"\nОТВЕТ:")
    print(result["answer"])

    print(f"\nМетаданные:")
    print(f"  • Найдено источников: {result['sources_count']}")
    print(f"  • Время выполнения: {result['execution_time']}с")

    if result["sources"]:
        print(f"\nИсточники:")
        for j, source in enumerate(result["sources"][:3], 1):
            print(f"  {j}. {source['metadata'].get('source', 'Unknown')} "
                  f"(score: {source['similarity_score']:.3f})")

print("\n" + "=" * 80)
print("ТЕСТИРОВАНИЕ ЗАВЕРШЕНО")
print("=" * 80)
```

🎓 ЧАСТЬ 3: АДАПТАЦИЯ ПОД СПЕЦИАЛЬНОСТЬ

Задание 3.1: Создание доменно-специфичной RAG системы

Инструкция: Адаптируйте RAG-систему под вашу научную специальность. Заполните документы в `data/documents/` релевантными материалами.

Таблица вариантов адаптации

Специальность	Тип документов	Примеры запросов	Особенности
Информационная безопасность	Стандарты, политики, отчёты об уязвимостях	"Какие требования PCI DSS?", "Уязвимости OWASP"	Точные цитаты важны
Автоматизация процессов	Технологические карты, инструкции	"Параметры процесса X", "Нормы времени"	Числовые данные
Строительство	СНиП, ГОСТ, проекты	"Нагрузки на фундамент", "Требования к бетону"	Нормативные ссылки
Энергетика	Схемы, регламенты, отчёты	"Режимы работы оборудования", "Нормы потребления"	Временные ряды
Транспорт	Маршруты, расписания, регламенты	"Оптимизация маршрута X", "Нормы расхода"	Геоданные
Программная инженерия	Документация кода, API	"Как использовать API X?", "Архитектура системы"	Примеры кода
Медицинская техника	Инструкции, протоколы	"Параметры оборудования", "Протоколы диагностики"	Точные значения

Листинг 3.1: Пример адаптации для ИБ

```python
# Файл: src/rag/security_rag_pipeline.py
# -*- coding: utf-8 -*-
"""
Специализированный RAG для информационной безопасности
Лабораторная работа №5

Автор: [Ваша фамилия]
Специальность: Информационная безопасность
"""

from rag.rag_pipeline import RAGPipeline
from typing import Dict, Any, List

class SecurityRAGPipeline(RAGPipeline):
    """
    RAG-система для вопросов по информационной безопасности.

    Особенности:
    • Приоритет нормативным документам (ГОСТ, ФСТЭК)
    • Точное цитирование требований
    • Классификация по уровням критичности
    """

    def __init__(self, vectorstore, llm=None, top_k: int = 5):
        super().__init__(vectorstore, llm, top_k)

        # Специфичный промпт для ИБ
        self.rag_prompt = ChatPromptTemplate.from_template("""
Ты — эксперт по информационной безопасности.
Используй ТОЛЬКО предоставленный контекст для ответа.
Цитируй конкретные пункты нормативных документов.

Если вопрос касается требований безопасности:
• Укажи нормативный документ (ГОСТ, ФСТЭК, PCI DSS)
• Приведи конкретный пункт/требование
• Укажи уровень критичности

Контекст из документов:
{context}

Вопрос: {question}

Ответ:
""")

    def query(self, question: str, include_sources: bool = True) -> Dict[str, Any]:
        """Запрос с ИБ-специфичной обработкой."""
        result = super().query(question, include_sources)

        # Добавление ИБ-метаданных
        result["domain"] = "information_security"
        result["compliance_check"] = self._check_compliance_keywords(question)

        return result

    def _check_compliance_keywords(self, question: str) -> Dict[str, bool]:
        """Проверка на ключевые слова комплаенса."""
```

```python
        keywords = {
            "gost": any(kw in question.lower() for kw in ["гост", "ГОСТ"]),
            "fstek": any(kw in question.lower() for kw in ["фстэк", "ФСТЭК"]),
            "pci_dss": any(kw in question.lower() for kw in ["pci dss", "pci_dss"]),
            "152fz": any(kw in question.lower() for kw in ["152-фз", "персональные данные"])
        }
        return keywords


# Использование
if __name__ == "__main__":
    # Инициализация как обычный RAGPipeline
    # Но с ИБ-специфичной логикой
    pass
```

📝 ЧАСТЬ 4: ТРЕБОВАНИЯ К ОТЧЁТУ

Структура отчёта (week5/docs/report.md)

```markdown
# Отчёт по лабораторной работе №5
## Дисциплина: Искусственный интеллект

---

## Общая информация

| Параметр | Значение |
|----------|----------|
| **Студент** | [ФИО полностью] |
| **Группа** | [Номер группы] |
| **Дата выполнения** | [ДД.ММ.ГГГГ] |
| **Специальность** | [Название специальности] |
| **Тема диплома** | [Тема дипломной работы] |

---

## 1. Цель работы

[Опишите цель работы своими словами]

---

## 2. Выполненные задачи

- [ ] Настроены зависимости LangChain + ChromaDB
- [ ] Реализована загрузка документов
- [ ] Настроено разбиение на чанки
- [ ] Создано векторное хранилище
- [ ] Реализован RAG пайплайн
- [ ] Выполнена адаптация под специальность
- [ ] Код загружен в GitHub

---

## 3. Ход работы

### 3.1. Архитектура RAG системы

[Схема архитектуры с пояснениями]

### 3.2. Настроенные компоненты

| Компонент | Реализация | Параметры |
|-----------|-----------|-----------|
| Document Loader | [Ваша реализация] | Форматы: ... |
| Chunking | [Стратегия] | Size: ..., Overlap: ... |
| Embeddings | [Модель] | Dimension: ... |
| Vector Store | ChromaDB | Collection: ... |
| LLM | YandexGPT/GigaChat | Temperature: ... |

### 3.3. Документы для индексации

[Описание загруженных документов]
[Статистика: количество, размер, типы]
```

```
### 3.4. Тестовые запросы и ответы

| № | Запрос | Ответ (кратко) | Источников | Время |
|---|--------|---------------|-----------|-------|
| 1 | ... | ... | ... | ... |
| 2 | ... | ... | ... | ... |
| 3 | ... | ... | ... | ... |

### 3.5. Адаптация под специальность

[Описание специализированной RAG]
[Примеры запросов по вашей теме]
[Интеграция с дипломом]

### 3.6. Проблемы и решения

[Описание возникших проблем]
[Как были решены]

---

## 4. Результаты

| Критерий | Статус |
|----------|--------|
| RAG работает | ✅ |
| Документы индексированы | ✅ |
| Поиск релевантный | ✅ |
| Специализация выполнена | ✅ |
| Код в GitHub | ✅ |

---

## 5. Выводы

[Что изучили, трудности, планы]

---

## 6. Список источников

. . .
```

🚀 ЧАСТЬ 5: ЗАГРУЗКА В GITHUB

```
# Переход в директорию репозитория
cd ~/ai-course-labs

# Проверка ветки
git branch  # Должно быть * week5

# Добавление файлов
git add week5/

# Коммит
git commit -m "Lab 5: RAG System with document processing

- Implemented document loading (PDF, TXT, MD)
- Created chunking pipeline with overlap
- Integrated ChromaDB for vector storage
- Built RAG pipeline with YandexGPT
- Adapted for specialty domain
- Added tests and documentation

Author: [ВАША ФАМИЛИЯ]
Group: [НОМЕР ГРУППЫ]"

# Отправка в GitHub
git push -u origin week5
```